📚 PHP & Laravel: A Complete Learning Guide

From Zero to Understanding Your Own Project

Written exclusively for: Md Zahid Hasan Nerob (ID: 24341207) CSE470 — Software Engineering, Section 9

⚠️ IMPORTANT: This book is designed around **YOUR actual project** code. Every concept is explained first in simple English, then shown with a real example from `cse470_project`. By the end, you will be able to explain any part of your project confidently during your viva.

Table of Contents

1. How the Web Works — The Big Picture
2. PHP Fundamentals — The Language
3. Object-Oriented PHP — Classes & Objects
4. Databases & SQL — Where Data Lives
5. MVC Architecture — The Design Pattern
6. Laravel — The Framework
7. Laravel Routing — How Requests Travel
8. Laravel Controllers — The Traffic Manager
9. Laravel Models & Eloquent — Talking to the Database
10. Laravel Migrations — Building Database Tables
11. Laravel Blade Views — What Users See
12. Service Classes — Keeping Business Logic Clean
13. Authentication & Sessions — Who Are You?
14. Database Transactions — All-or-Nothing Safety
15. Your Project — Feature-by-Feature Walkthrough
16. Viva Q&A Master Sheet

Chapter 1: How the Web Works — The Big Picture

Before touching any code, you need to understand what actually happens when someone opens a website.

1.1 The Client-Server Model

Imagine a **restaurant** (this is a perfect metaphor for your project!):

Real Restaurant	Web Application
Customer orders food	**Browser** sends a request
Waiter takes the order	**Web Server** receives it
Kitchen prepares food	**PHP/Laravel** processes it
Menu lists dishes	**Database** stores all data
Plate of food comes back	**HTML Page** comes back

When you type `http://127.0.0.1:8000/sales` in your browser:

```
Your Browser          Your Computer (Server)
─────────────         ──────────────────────────

                      Laravel App (PHP)
"Give me /sales" ──►  Route: found sales.index
                      Controller: SaleController@index
                      Model: Sale::all() → Database
                      View: sales/index.blade.php
"Here's the page" ◄──  HTML generated and sent back
```

1.2 HTTP — The Language of the Web

Browsers and servers talk using **HTTP (HyperText Transfer Protocol)**. There are two main types of messages:

GET Request — "I want to SEE something"

```
GET /sales HTTP/1.1
Host: 127.0.0.1:8000
```

> When you click a link or type a URL, it's always a GET request.

POST Request — "I want to SEND/SUBMIT something"

```
POST /sales HTTP/1.1
Host: 127.0.0.1:8000
Content-Type: application/x-www-form-urlencoded

dish_id=3&quantity=2&notes=table+5
```

> When you submit a form (like recording a sale), it's a POST request.

HTTP Response Codes — what the server replies:

Code	Meaning	Your Project Example
`200 OK`	Everything worked	Viewing the sales list
`302 Found`	Redirect to another page	After saving a sale, go to list
`404 Not Found`	Page doesn't exist	Wrong URL
`500 Server Error`	PHP code crashed	Bug in your code

1.3 URLs Explained

```
http://127.0.0.1:8000/purchase-orders/5/approve
|       |         |    |               |
|       |         |    |               └── Action (approve this order)
|       |         |    └── Route path
|       |         └── Port number (8000 = development server)
|       └── IP Address (127.0.0.1 = "this same computer")
└── Protocol (http = web)
```

Chapter 2: PHP Fundamentals — The Language

PHP stands for **PHP: Hypertext Preprocessor**. It runs on the server (not in the browser) and generates HTML pages.

2.1 Basic PHP Syntax

Every PHP file starts with `<?php`. That's how the server knows "this is PHP code, execute it."

```
<?php
// This is a comment — the computer ignores it

echo "Hello, World!"; // Print something to the screen
```

2.2 Variables

Variables store data temporarily. In PHP, all variables start with `$`.

```
<?php
$name = "Zahid";          // String (text)
$age = 22;                 // Integer (whole number)
$price = 280.50;           // Float (decimal number)
$isManager = true;         // Boolean (true/false)
$items = ["Rice", "Oil"];  // Array (list of things)

echo $name;    // Prints: Zahid
echo $age;     // Prints: 22
```

> **In Your Project:**
>
> ```
> // From SaleService.php
> $deductQty = (float)$recipeItem->quantity * $quantity;
> // $deductQty is a float variable holding, e.g., 0.4 kg × 2 = 0.8 kg
> ```

2.3 Strings

```
<?php
$dishName = "Beef Burger";
$price = 280;

// Concatenation with . (dot)
echo "Dish: " . $dishName . " costs BDT " . $price;

// Double-quoted strings allow variables inside them
echo "Dish: {$dishName} costs BDT {$price}";

// Both output: Dish: Beef Burger costs BDT 280
```

2.4 Arrays

Arrays are like a list. They can hold multiple values.

```
<?php
// Simple indexed array (numbered from 0)
$ingredients = ["Beef Patty", "Burger Bun", "Tomato"];
echo $ingredients[0]; // Beef Patty
echo $ingredients[1]; // Burger Bun

// Associative array (key => value pairs, like a dictionary)
$sale = [
    'dish_id'      => 3,
    'quantity'     => 2,
    'total_amount' => 560.00,
    'notes'        => 'Table 5',
];
echo $sale['dish_id'];      // 3
echo $sale['total_amount']; // 560.00
```

In Your Project:

```
// From SaleService.php — this is an associative array being saved to database
$sale = Sale::create([
    'dish_id'      => $dish->id,
    'quantity'     => $quantity,
    'unit_price'   => $dish->price,
    'total_amount' => $totalAmount,
    'recorded_by'  => $recordedBy,
    'notes'        => $notes,
]);
```

2.5 If / Else Conditions

```
<?php
$stock = 5.0;
$required = 8.0;

if ($stock >= $required) {
    echo "Enough stock!";
} elseif ($stock > 0) {
    echo "Some stock, but not enough.";
} else {
    echo "No stock at all!";
}
// Prints: Some stock, but not enough.
```

In Your Project:

```
// From Ingredient.php model
public function getStockStatusAttribute(): string
{
    if ($this->usable_stock <= 0) return 'out_of_stock';
    if ($this->usable_stock <= (float)$this->reorder_threshold) return 'low';
    return 'ok';
}
```

This checks the stock level and returns a status string.

2.6 Loops

foreach — go through every item in an array:

```php
<?php
$batches = ["BAT-001", "BAT-002", "BAT-003"];

foreach ($batches as $batch) {
    echo "Processing: " . $batch . "\n";
}
// Processing: BAT-001
// Processing: BAT-002
// Processing: BAT-003
```

> **In Your Project:**
>
> ```php
> // From SaleService.php — deduct from each ingredient in the recipe
> foreach ($dish->recipeIngredients as $recipeItem) {
>     $ingredient = $recipeItem->ingredient;
>     $deductQty = (float)$recipeItem->quantity * $quantity;
>     // ... deduct from stock
> }
> ```
>
> This loops through **every ingredient** in the dish's recipe and deducts the required amount.

2.7 Functions

Functions are reusable blocks of code.

```php
<?php
// Define a function
function calculateTotal($unitPrice, $quantity) {
    return $unitPrice * $quantity;  // "return" sends a value back
}

// Call the function
$total = calculateTotal(280, 3);
echo $total; // 840
```

2.8 Null and Type Casting

```php
<?php
$notes = null;  // "nothing" / empty

if ($notes === null) {
    echo "No notes provided";
}

// Type casting — force a variable to be a specific type
$stringNumber = "42.5";
$float = (float)$stringNumber;  // Now it's a number: 42.5
$integer = (int)$stringNumber;  // Now it's 42 (truncated)
```

> **In Your Project:**
>
> ```
> // (float)$recipeItem->quantity — cast to float to ensure decimal math works
> $deductQty = (float)$recipeItem->quantity * $quantity;
> ```

Chapter 3: Object-Oriented PHP — Classes & Objects

This is the most important concept for understanding Laravel. Everything in Laravel is built with **classes and objects**.

3.1 What is a Class?

A **class** is a blueprint/template. An **object** is a real thing made from that blueprint.

```
CLASS (Blueprint)          OBJECT (Real Thing)
─────────────────          ──────────────────

Ingredient                 Chicken Breast
  - name                     - name: "Chicken Breast"
  - unit                     - unit: "kg"
  - current_stock            - current_stock: 15.0
  - cost_per_unit            - cost_per_unit: 350.00
```

3.2 Defining a Class

```
<?php

class Ingredient {
    // Properties (variables belonging to this class)
    public $name;
    public $unit;
    public $current_stock;
    public $cost_per_unit;

    // Constructor — runs when you create a new object
    public function __construct($name, $unit, $stock, $cost) {
        $this->name = $name;         // $this means "this object"
        $this->unit = $unit;
        $this->current_stock = $stock;
        $this->cost_per_unit = $cost;
    }

    // Method (function belonging to this class)
    public function isLowStock($threshold) {
        return $this->current_stock <= $threshold;
    }

    public function getValue() {
        return $this->current_stock * $this->cost_per_unit;
    }
}

// Creating objects (instances) from the class
$chicken = new Ingredient("Chicken Breast", "kg", 15.0, 350.00);
$oil = new Ingredient("Cooking Oil", "litre", 8.0, 180.00);

echo $chicken->name;              // Chicken Breast
echo $chicken->isLowStock(10);    // false (15 > 10)
echo $oil->getValue();            // 1440.00
```

3.3 `$this` — The Most Important Keyword

Inside a class, `$this` always refers to **the current object** you're working with.

```
class Sale {
    public $quantity;
    public $unit_price;

    public function getTotalAmount() {
        // $this->quantity = this sale's quantity
        // $this->unit_price = this sale's unit price
        return $this->quantity * $this->unit_price;
    }
}
```

3.4 Inheritance — Building on Top of Classes

One class can **extend** (inherit from) another class, getting all its properties and methods.

```php
<?php

class Model {
    // Laravel's base Model has: save(), create(), find(), etc.
    public function save() {
        echo "Saving to database...";
    }
}

class Ingredient extends Model {
    // Ingredient INHERITS save() from Model
    // Plus it adds its own methods
    public function isLowStock($threshold) {
        return $this->current_stock <= $threshold;
    }
}

$ingredient = new Ingredient();
$ingredient->save();         // Works! Inherited from Model
$ingredient->isLowStock(5); // Works! Defined in Ingredient
```

> **In Your Project:**
>
> ```php
> // From app/Models/Ingredient.php
> class Ingredient extends Model   // ← extends Laravel's Model class
> {
>     // ...your code...
> }
> ```
>
> `Ingredient` inherits **all database powers** from Laravel's `Model` class.

3.5 Namespaces — Organizing Code into Folders

Namespaces prevent name conflicts. Think of them like folder paths.

```php
<?php

namespace App\Models;  // This file lives in app/Models/ folder

class Ingredient extends Model {
    // ...
}
```

```php
<?php

namespace App\Http\Controllers;  // This file lives in app/Http/Controllers/ folder

use App\Models\Ingredient;  // "import" Ingredient from its namespace

class IngredientController extends Controller {
    public function index() {
        $ingredients = Ingredient::all(); // Use it here
    }
}
```

3.6 Access Modifiers

```php
class MyClass {
    public $name;    // Anyone can access this
    protected $age;  // Only this class and child classes
    private $secret; // Only this class itself
}
```

3.7 Static Methods

Static methods belong to the **class itself**, not an object. You call them with `::`.

```php
// Regular method — needs an object first
$sale = new Sale();
$sale->save();

// Static method — called directly on the class
$allSales = Sale::all();    // No "new" needed
$oneSale = Sale::find(5);   // Find sale with ID 5
```

> **In Your Project:**
>
> ```php
> // From SaleController.php
> $totalRevenue = Sale::sum('total_amount');  // Static — no object needed
> $totalOrders  = Sale::count();              // Static — no object needed
> ```

Chapter 4: Databases & SQL — Where Data Lives

Your project stores everything in a **SQLite database** file at `database/database.sqlite`.

4.1 What is a Database?

A database is a structured collection of data organized into **tables**. Think of it like a highly organized collection of Excel spreadsheets.

```
DATABASE: restaurant_supply_hub
├── Table: users
├── Table: ingredients
├── Table: ingredient_batches
├── Table: dishes
├── Table: recipe_ingredients
├── Table: sales
├── Table: sale_deductions
├── Table: purchase_orders
├── Table: purchase_order_items
├── Table: stock_reconciliations
├── Table: stock_reconciliation_items
└── Table: waste_reports
```

4.2 Tables, Rows, and Columns

id	name	unit	current_stock	reorder_threshold	cost_per_unit
1	Chicken Breast	kg	15.00	10.00	350.00
2	Beef Patty	kg	8.00	5.00	420.00
3	Burger Bun	piece	50	20	15.00

- **Table**: `ingredients` — stores all ingredients
- **Column**: `name`, `unit`, `current_stock` — categories of data
- **Row**: Each row is ONE ingredient's data

4.3 Primary Keys & Foreign Keys

```
Table: dishes          Table: recipe_ingredients
──────────────         ─────────────────────────
id: 1 ◄──────────────── dish_id: 1
name: Beef Burger      ingredient_id: 2 ──────── Table: ingredients
price: 280             quantity: 0.2             id: 2
                                                 name: Beef Patty
```

- **Primary Key (`id`)**: Unique identifier for each row
- **Foreign Key (`dish_id`, `ingredient_id`)**: Links rows between tables

4.4 Basic SQL Queries

Even though Laravel writes SQL for you, knowing what it generates helps you understand your code:

```
-- SELECT: Get data
SELECT * FROM ingredients;
SELECT name, current_stock FROM ingredients WHERE current_stock < 10;

-- INSERT: Add new data
INSERT INTO sales (dish_id, quantity, total_amount) VALUES (1, 2, 560);

-- UPDATE: Change existing data
UPDATE ingredients SET current_stock = 12.5 WHERE id = 1;

-- DELETE: Remove data
DELETE FROM waste_reports WHERE id = 5;
```

4.5 Relationships Between Tables

One-to-Many: One dish has MANY recipe ingredients

```
dishes (1)  ──────►  recipe_ingredients (many)
   id: 1                 dish_id: 1,  ingredient_id: 2, quantity: 0.2
                          dish_id: 1,  ingredient_id: 3, quantity: 2
```

Many-to-Many: Many dishes use many ingredients (via pivot table)

```
dishes ◄──────  recipe_ingredients ──────► ingredients
  id: 1            (pivot table)             id: 2 (Beef Patty)
  id: 2                                      id: 3 (Burger Bun)
```

Chapter 5: MVC Architecture — The Design Pattern

This is the **most critical concept** for your viva. Your faculty SADA will definitely ask about this.

5.1 What is MVC?

MVC = Model + View + Controller

It's a way to **organize your code** so that:

- Data logic is in one place
- Display logic is in another place
- Request handling is in a third place

5.2 The Restaurant Analogy

MVC Component	Restaurant Equivalent	Your Project
Model	The kitchen & recipe book	`app/Models/` — talks to database
View	The dining room & menu	`resources/views/` — HTML shown to user
Controller	The waiter	`app/Http/Controllers/` — connects Model and View

5.3 How a Request Flows Through MVC

```
1. User clicks "Record Sale" button
       ↓
2. Browser sends POST /sales to server
       ↓
3. ROUTER: "POST /sales → SaleController@store"
       ↓
4. CONTROLLER (SaleController::store):
   - Validates input (dish_id, quantity)
   - Calls SaleService->recordSale()
       ↓
5. SERVICE (SaleService::recordSale):
   - Business logic: check stock, deduct, save
       ↓
6. MODEL (Sale, Ingredient, SaleDeduction):
   - Actually reads/writes the database
       ↓
7. CONTROLLER: Gets result, redirects to sales.index
       ↓
8. VIEW (sales/index.blade.php):
   - Renders the HTML table of all sales
       ↓
9. Browser shows the HTML page to user
```

5.4 Why MVC? (Answer for Viva)

> **"Why did you use Service classes instead of putting business logic in the Controller?"**

Answer:

- **Without MVC**: Imagine mixing your cooking, serving, and billing all done by one person. Chaotic.
- **With MVC**: Each person has ONE job. If the recipe changes, only the kitchen is affected.
- **Service classes** keep the **business rules** (like "check stock before deducting") separate from the **HTTP handling** (like "which URL called this"). This makes testing easier and the code more maintainable.

Chapter 6: Laravel — The Framework

6.1 What is a Framework?

A framework is a **set of pre-built code** that handles common tasks so you don't have to write everything from scratch.

Without Laravel (raw PHP):

```
$pdo = new PDO("sqlite:database.sqlite");
$stmt = $pdo->prepare("SELECT * FROM ingredients WHERE id = ?");
$stmt->execute([$id]);
$ingredient = $stmt->fetch(PDO::FETCH_ASSOC);
```

With Laravel (Eloquent ORM):

```
$ingredient = Ingredient::find($id);
```

6.2 Laravel's Folder Structure

```
cse470_project/
├── app/                        ← Your main PHP code
│   ├── Http/
│   │   └── Controllers/        ← Controllers (C in MVC)
│   │       ├── SaleController.php
│   │       ├── BatchController.php
│   │       └── ...
│   ├── Models/                 ← Models (M in MVC)
│   │   ├── Ingredient.php
│   │   ├── Sale.php
│   │   └── ...
│   └── Services/               ← Business logic (extra layer)
│       ├── SaleService.php
│       └── ...
│
├── config/                     ← Configuration files
├── database/
│   ├── migrations/             ← Database table definitions
│   ├── seeders/                ← Test data
│   └── database.sqlite         ← Your actual database file
│
├── resources/
│   └── views/                  ← Blade template files (V in MVC)
│       ├── layouts/
│       │   └── app.blade.php ← Master layout (navbar, footer)
│       ├── sales/
│       │   ├── index.blade.php
│       │   └── create.blade.php
│       └── ...
│
├── routes/
│   └── web.php                 ← URL definitions
│
└── artisan                     ← Laravel's command-line tool
```

6.3 Artisan — Laravel's Command-Line Tool

```
php artisan serve           # Start the development server
php artisan make:controller SaleController   # Create controller
php artisan make:model Sale  # Create model
php artisan migrate          # Run database migrations
php artisan db:seed          # Seed database with test data
php artisan route:list       # List all routes
```

Chapter 7: Laravel Routing — How Requests Travel

The **router** is the first thing that handles every request. It reads the URL and decides which controller should handle it.

7.1 Basic Route Definitions

Your routes are defined in `routes/web.php`:

```
<?php
use App\Http\Controllers\SaleController;
use Illuminate\Support\Facades\Route;

// GET request to /sales → call SaleController's index method
Route::get('/sales', [SaleController::class, 'index'])->name('sales.index');

// POST request to /sales → call SaleController's store method
Route::post('/sales', [SaleController::class, 'store'])->name('sales.store');

// GET request to /sales/create → call SaleController's create method
Route::get('/sales/create', [SaleController::class, 'create'])->name('sales.create');
```

7.2 Route with Parameters

```
// {id} is a variable — it captures the number from the URL
// URL: /purchase-orders/5/approve → $id = 5
Route::post('purchase-orders/{id}/approve', [PurchaseOrderController::class, 'approve'])
    ->name('purchase-orders.approve');
```

7.3 Resource Routes (Shortcut)

Instead of writing all routes manually, `Route::resource` creates 7 standard routes at once:

```
Route::resource('sales', SaleController::class)->only(['index', 'create', 'store']);
```

This creates:

Method	URL	Controller Method	Route Name
GET	`/sales`	`index()`	`sales.index`
GET	`/sales/create`	`create()`	`sales.create`
POST	`/sales`	`store()`	`sales.store`

7.4 Named Routes

Naming routes lets you generate URLs in your code without hardcoding paths:

```
route('sales.index')       // Returns: http://127.0.0.1:8000/sales
route('purchase-orders.approve', ['id' => 5])  // Returns: .../purchase-orders/5/approve

// Use in Blade links
<a href="{{ route('sales.create') }}">Record New Sale</a>
```

7.5 Middleware

Middleware is code that runs **before** your controller. Like a security guard at the door.

```
// 'web' middleware group handles sessions and CSRF protection
Route::middleware('web')->group(function () {
    Route::get('/', [DashboardController::class, 'index'])->name('dashboard');
});
```

Chapter 8: Laravel Controllers — The Traffic Manager

Controllers receive HTTP requests, do minimal work, and return responses.

8.1 Basic Controller Structure

```php
<?php

namespace App\Http\Controllers;

use App\Models\Sale;
use App\Services\SaleService;
use Illuminate\Http\Request;

class SaleController extends Controller
{
    // Constructor — called when this controller is created
    // Laravel automatically creates SaleService and passes it here (Dependency Injection)
    public function __construct(private SaleService $saleService) {}

    // Handles GET /sales
    public function index()
    {
        $sales = Sale::with(['dish', 'recorder'])->latest()->paginate(15);
        return view('sales.index', compact('sales'));
    }

    // Handles GET /sales/create
    public function create()
    {
        $dishes = Dish::where('is_active', true)->get();
        return view('sales.create', compact('dishes'));
    }

    // Handles POST /sales
    public function store(Request $request)
    {
        // 1. Validate the submitted form data
        $request->validate([
            'dish_id'  => 'required|exists:dishes,id',
            'quantity' => 'required|integer|min:1|max:100',
            'notes'    => 'nullable|string|max:500',
        ]);

        try {
            // 2. Do the work (via Service)
            $sale = $this->saleService->recordSale(
                $request->dish_id,
                $request->quantity,
                Auth::id()
            );
            // 3. Return a response (redirect)
            return redirect()->route('sales.index')->with('success', 'Sale recorded!');
        } catch (Exception $e) {
            return back()->withInput()->withErrors(['stock_error' => $e->getMessage()]);
        }
    }
}
```

8.2 Validation Rules

```
$request->validate([
    'dish_id'  => 'required|exists:dishes,id',
    // 'required' = must not be empty
    // 'exists:dishes,id' = must be an ID that exists in the dishes table

    'quantity' => 'required|integer|min:1|max:100',
    // 'integer' = must be a whole number
    // 'min:1' = must be at least 1
    // 'max:100' = must be 100 or less

    'notes' => 'nullable|string|max:500',
    // 'nullable' = can be empty/null
]);
// If validation fails: automatically redirects back with error messages
// If validation passes: code continues below
```

8.3 Returning Responses

```
// Return a view
return view('sales.index', compact('sales'));

// Redirect to another route
return redirect()->route('sales.index');

// Redirect with a flash message
return redirect()->route('sales.index')->with('success', 'Sale recorded!');

// Redirect back with errors
return back()->withInput()->withErrors(['error' => 'Something went wrong']);
```

Chapter 9: Laravel Models & Eloquent — Talking to the Database

Eloquent is Laravel's ORM (Object-Relational Mapper). It lets you work with database rows as PHP objects, without writing SQL.

9.1 A Model Represents a Table

```
// The Ingredient class represents the "ingredients" table
// Laravel automatically assumes the table name is "ingredients"
class Ingredient extends Model
{
    // ...
}
```

9.2 The `$fillable` Property — Security Feature

```
class Ingredient extends Model
{
    protected $fillable = [
        'name',
        'unit',
        'current_stock',
        'reorder_threshold',
        'target_stock',
        'cost_per_unit',
        'category',
    ];
    // Only these fields can be set via create() or update()
    // Other fields (like 'id', 'created_at') are protected
}
```

9.3 Basic Eloquent Operations (CRUD)

```
// CREATE — Insert a new row
$ingredient = Ingredient::create([
    'name' => 'Tomato',
    'unit' => 'kg',
    'current_stock' => 10.0,
]);

// READ — Get rows from database
$all    = Ingredient::all();           // All ingredients
$one    = Ingredient::find(3);         // Ingredient with id=3
$low    = Ingredient::where('current_stock', '<', 5)->get(); // With condition
$paged  = Ingredient::paginate(15);    // 15 per page

// UPDATE — Change existing data
$ingredient = Ingredient::find(1);
$ingredient->current_stock = 12.5;
$ingredient->save();

// DELETE
$ingredient = Ingredient::find(5);
$ingredient->delete();
```

9.4 Relationships

hasMany (One-to-Many)

```
// In Ingredient model: One ingredient has MANY batches
public function batches()
{
    return $this->hasMany(IngredientBatch::class);
    // Laravel finds batches using ingredient_id column in batches table
}

// Usage:
$ingredient = Ingredient::find(1);
$batches = $ingredient->batches;  // Gets all batches for this ingredient
```

belongsTo (Many-to-One / inverse)

```
// In Sale model: Each sale belongs to ONE dish
public function dish()
{
    return $this->belongsTo(Dish::class);
    // Laravel uses dish_id column in sales table
}

// Usage:
$sale = Sale::find(1);
$dishName = $sale->dish->name;  // "Beef Burger"
```

belongsToMany (Many-to-Many)

```
// In Ingredient model: Ingredients can be in MANY dishes
public function dishes()
{
    return $this->belongsToMany(Dish::class, 'recipe_ingredients');
    // 'recipe_ingredients' is the pivot table
}
```

9.5 Eager Loading (with)

```
// BAD - N+1 problem (100 sales = 101 queries!)
$sales = Sale::all();
foreach ($sales as $sale) {
    echo $sale->dish->name; // Each iteration queries the dishes table
}

// GOOD - Eager loading (2 queries total)
$sales = Sale::with(['dish', 'recorder', 'deductions.ingredient'])->get();
foreach ($sales as $sale) {
    echo $sale->dish->name; // Already loaded, no extra queries
}
```

> **From Your Project:**
>
> ```
> // SaleController.php
> $sales = Sale::with(['dish', 'recorder', 'deductions.ingredient'])->latest()-
> >paginate(15);
> // Loads: each sale's dish, the user who recorded it, and all ingredient deductions
> ```

9.6 Accessors (Custom Attributes)

Accessors let you add computed properties that don't exist as database columns:

```
// In Ingredient.php
// Method named get[AttributeName]Attribute
public function getUsableStockAttribute(): float
{
    // Calculate usable stock by subtracting expired batch amounts
    $expiredStock = $this->batches()
        ->whereNotNull('expiry_date')
        ->where('expiry_date', '<', now()->toDateString())
        ->sum('remaining_quantity');

    return max(0, (float)$this->current_stock - (float)$expiredStock);
}

// Usage - accessed like a regular property:
$ingredient->usable_stock  // Calls the accessor automatically
```

> **Viva Answer:** *"The `usable_stock` property is an Eloquent accessor — it's not a column in the database, but a computed value. Every time `$ingredient->usable_stock` is accessed, PHP calls `getUsableStockAttribute()` which queries the batches table to find expired quantities and subtracts them from the total current stock."*

9.7 The `$casts` Property

```
protected $casts = [
    'current_stock'     => 'decimal:2',  // Always stored as 2-decimal float
    'reorder_threshold' => 'decimal:2',
];
// Without this, the database might return "10" (string) instead of 10.00 (float)
```

Chapter 10: Laravel Migrations — Building Database Tables

Migrations are PHP files that define your database structure.

10.1 Why Migrations?

- **Version control**: Everyone on the team runs the same migrations → same database structure.
- **Reversible**: You can undo a migration.
- **Documented**: Database structure is documented in code.

10.2 A Migration File

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    // "up" = create the table (run when migrating)
    public function up(): void
    {
        Schema::create('ingredients', function (Blueprint $table) {
            $table->id();                          // Auto-increment integer primary key
            $table->string('name');                // VARCHAR(255) column
            $table->string('unit');
            $table->decimal('current_stock', 10, 2)->default(0);
            $table->decimal('reorder_threshold', 10, 2)->default(0);
            $table->decimal('target_stock', 10, 2)->default(0);
            $table->decimal('cost_per_unit', 10, 2)->default(0);
            $table->string('category')->nullable(); // Can be empty
            $table->timestamps();                  // created_at and updated_at columns
        });
    }

    // "down" = undo the table creation (run when rolling back)
    public function down(): void
    {
        Schema::dropIfExists('ingredients');
    }
};
```

10.3 Foreign Keys (Linking Tables)

```
Schema::create('sales', function (Blueprint $table) {
    $table->id();
    $table->foreignId('dish_id')->constrained()->onDelete('cascade');
    // ↑ Creates dish_id column + foreign key constraint linking to dishes.id
    // onDelete('cascade') = if the dish is deleted, delete related sales too

    $table->foreignId('recorded_by')->constrained('users');
    // ↑ Foreign key to users table, but column is named 'recorded_by'

    $table->integer('quantity');
    $table->decimal('total_amount', 10, 2);
    $table->timestamps();
});
```

10.4 Running Migrations

```
php artisan migrate             # Run all pending migrations
php artisan migrate:rollback    # Undo the last batch
php artisan migrate:fresh       # Drop all tables and re-run everything
php artisan migrate:status      # Show which migrations have been run
```

10.5 Seeders — Filling Tables with Test Data

```
class DatabaseSeeder extends Seeder
{
    public function run(): void
    {
        User::create([
            'name' => 'Restaurant Manager',
            'email' => 'manager@restaurant.com',
            'password' => bcrypt('password'),
            'role' => 'manager'
        ]);

        Ingredient::create([
            'name' => 'Chicken Breast',
            'unit' => 'kg',
            'current_stock' => 15.0,
            'reorder_threshold' => 10.0
        ]);
    }
}
```

```
php artisan db:seed             # Run seeders
php artisan migrate:fresh --seed # Drop, re-create, and seed (complete reset)
```

Chapter 11: Laravel Blade Views — What Users See

Blade is Laravel's templating engine. It's HTML with special PHP-like syntax.

11.1 Displaying Data

```
{{-- This is a Blade comment --}}

{{-- Display a variable (auto-escaped for security) --}}
{{ $sale->total_amount }}

{{-- Display with a default if null --}}
{{ $sale->notes ?? 'No notes' }}
```

11.2 Control Structures

```
@if($ingredient->usable_stock <= 0)
    <span class="badge bg-danger">Out of Stock</span>
@elseif($ingredient->usable_stock <= $ingredient->reorder_threshold)
    <span class="badge bg-warning">Low Stock</span>
@else
    <span class="badge bg-success">In Stock</span>
@endif

@foreach($ingredients as $ingredient)
    <tr>
        <td>{{ $ingredient->name }}</td>
        <td>{{ $ingredient->current_stock }} {{ $ingredient->unit }}</td>
    </tr>
@endforeach

@forelse($sales as $sale)
    <tr><td>{{ $sale->dish->name }}</td></tr>
@empty
    <tr><td colspan="4">No sales recorded yet.</td></tr>
@endforelse
```

11.3 Layout Inheritance — The Master Template

Your project has a master layout at `resources/views/layouts/app.blade.php`:

```
{{-- layouts/app.blade.php --}}
<!DOCTYPE html>
<html>
<head>
    <title>@yield('title') — Restaurant Supply Hub</title>
    <!-- Bootstrap CSS loaded here once for ALL pages -->
</head>
<body>
    <!-- Navigation bar shown on ALL pages -->
    <nav>...</nav>

    <!-- Main content area - each page fills this in -->
    <main>
        @yield('content')
    </main>
</body>
</html>
```

A page **extends** the layout and fills in the `@yield` sections:

```
{{-- sales/index.blade.php --}}
@extends('layouts.app')

@section('title', 'Sales Records')

@section('content')
    <h1>Sales Records</h1>
    <table>
        @foreach($sales as $sale)
            <tr>
                <td>{{ $sale->dish->name }}</td>
                <td>{{ $sale->quantity }}</td>
                <td>৳{{ number_format($sale->total_amount, 2) }}</td>
            </tr>
        @endforeach
    </table>
@endsection
```

11.4 Forms in Blade

```
<form method="POST" action="{{ route('sales.store') }}">
    @csrf  {{-- CSRF protection token — REQUIRED for all POST forms --}}

    <select name="dish_id">
        @foreach($dishes as $dish)
            <option value="{{ $dish->id }}">{{ $dish->name }}</option>
        @endforeach
    </select>

    <input type="number" name="quantity" value="{{ old('quantity') }}" min="1">
    {{-- old('quantity') re-fills the field if form validation fails --}}

    <button type="submit">Record Sale</button>
</form>
```

11.5 Displaying Validation Errors

```
{{-- Show specific error for a field --}}
@error('quantity')
    <div class="alert alert-danger">{{ $message }}</div>
@enderror

{{-- Show custom error from stock exception --}}
@if($errors->has('stock_error'))
    <div class="alert alert-danger">{{ $errors->first('stock_error') }}</div>
@endif
```

11.6 Flash Messages

```
// Controller sends a flash message
return redirect()->route('sales.index')->with('success', 'Sale recorded!');
```

```
{{-- View displays it --}}
@if(session('success'))
    <div class="alert alert-success">{{ session('success') }}</div>
@endif
```

11.7 Passing Data from Controller to View

```
// Controller
$sales = Sale::all();
$totalRevenue = Sale::sum('total_amount');

return view('sales.index', compact('sales', 'totalRevenue'));
// compact('sales', 'totalRevenue') = ['sales' => $sales, 'totalRevenue' => $totalRevenue]
```

```
{{-- View: now $sales and $totalRevenue are available --}}
<p>Total Revenue: ৳{{ number_format($totalRevenue, 2) }}</p>
```

Chapter 12: Service Classes — Keeping Business Logic Clean

Your project has a `app/Services/` folder. This is NOT standard Laravel — it's a **best practice** you chose to follow.

12.1 Why Services?

```
WITHOUT services (bad):                    WITH services (good):
──────────────────────                     ──────────────────────
SaleController::store()                    SaleController::store()
  - validate input                           - validate input
  - check stock availability                 - call SaleService->recordSale()
  - run DB transaction                       - return response
  - save Sale record                         [20 lines total]
  - loop through recipe items
  - deduct from ingredient                 SaleService::recordSale()
  - deduct from batches (FIFO)              - check stock availability
  - create SaleDeduction record              - run DB transaction
  - return response                          - save Sale record
  [200 lines!]                               - loop through recipe items
                                             - deduct from ingredient
Controller is 200 lines!                   - deduct from batches (FIFO)
Hard to test, hard to modify               - create SaleDeduction record
                                           [80 lines - independently testable]
```

12.2 Dependency Injection in Controllers

The controller gets the service automatically:

```
class SaleController extends Controller
{
    // Laravel's Service Container automatically creates SaleService
    // and passes it to the constructor
    public function __construct(private SaleService $saleService) {}

    public function store(Request $request)
    {
        $sale = $this->saleService->recordSale(...);
    }
}
```

Chapter 13: Authentication & Sessions — Who Are You?

13.1 What is Authentication?

Authentication = verifying who you are (login). The system knows you are "Manager" and shows manager-specific options.

13.2 Sessions

HTTP is "stateless" — the server forgets you after each request. **Sessions** solve this by storing a small ID in a cookie:

```
Request 1:  User logs in → Server creates session → Sends cookie with session ID
Request 2:  Browser sends cookie → Server reads session → "Oh, it's the manager!"
```

13.3 Your Auth Controller

```
// From AuthController.php
public function login(Request $request)
{
    $credentials = $request->validate([
        'email'    => 'required|email',
        'password' => 'required',
    ]);

    if (Auth::attempt($credentials)) {
        // Password matched → create session
        $request->session()->regenerate();  // Security: new session ID
        return redirect()->intended('/');   // Go to dashboard
    }

    return back()->withErrors(['email' => 'Invalid credentials']);
}

public function logout(Request $request)
{
    Auth::logout();                          // Clear session
    $request->session()->invalidate();       // Destroy session data
    $request->session()->regenerateToken();  // New CSRF token
    return redirect()->route('login');
}

// Quick 1-click login for demo purposes
public function quickLogin(string $role)
{
    $user = User::where('role', $role)->first();
    Auth::login($user);
    return redirect('/');
}
```

13.4 Using Auth in Controllers

```
use Illuminate\Support\Facades\Auth;

$user = Auth::user();     // Get the currently logged-in user
$userId = Auth::id();     // Get the current user's ID

if (Auth::check()) {
    echo "You are logged in as " . Auth::user()->name;
}
```

13.5 Role-Based Access

```
// In controllers:
if (Auth::user()->role === 'manager') {
    // Show approve button
}
```

```
{{-- In Blade views --}}
@if(auth()->user() && auth()->user()->role === 'manager')
    <button>Approve</button>
@endif
```

13.6 CSRF Protection

CSRF (Cross-Site Request Forgery) is an attack where a malicious website tricks your browser into submitting a form to your app. Laravel prevents this with tokens:

```
<form method="POST" action="/sales">
    @csrf  {{-- Generates hidden input with a unique secret token --}}
    <!-- form fields -->
</form>
```

Every POST form **must** have `@csrf` or Laravel will reject it with a 419 error.

Chapter 14: Database Transactions — All-or-Nothing Safety

14.1 Why Transactions?

Imagine recording a sale requires 3 database operations:

1. Create sale record
2. Deduct stock from ingredient
3. Update batch remaining quantity

What if step 2 succeeds but step 3 fails? You'd have corrupted data!

```
Without Transaction:
  ✅ Create sale record
  ✅ Deduct ingredient stock
  ❌ Update batch → ERROR!
  → Database is now inconsistent!

With Transaction:
  ✅ Create sale record
  ✅ Deduct ingredient stock
  ❌ Update batch → ERROR!
  → ROLLBACK: Undo steps 1 and 2
  → Database is clean, like nothing happened
```

14.2 DB::transaction in Your Project

```
// From SaleService.php
return DB::transaction(function () use ($dish, $quantity, $recordedBy, $notes) {
    // Step 1: Create the sale record
    $sale = Sale::create([
        'dish_id' => $dish->id,
        'quantity' => $quantity,
        'total_amount' => $totalAmount,
        'recorded_by' => $recordedBy,
    ]);

    // Step 2: For each recipe ingredient, deduct stock
    foreach ($dish->recipeIngredients as $recipeItem) {
        $ingredient = $recipeItem->ingredient;
        $deductQty = (float)$recipeItem->quantity * $quantity;

        // Step 3: Log the deduction
        SaleDeduction::create([
            'sale_id' => $sale->id,
            'ingredient_id' => $ingredient->id,
            'quantity_deducted' => $deductQty,
        ]);

        // Step 4: Reduce ingredient stock
        $ingredient->current_stock -= $deductQty;
        $ingredient->save();

        // Step 5: Deduct from batches (FIFO)
        $remainingToDeduct = $deductQty;
        $batches = $ingredient->batches()
            ->where('remaining_quantity', '>', 0)
            ->orderByRaw('CASE WHEN expiry_date IS NULL THEN 1 ELSE 0 END, expiry_date ASC')
            ->get();

        foreach ($batches as $batch) {
            if ($remainingToDeduct <= 0) break;
            $take = min((float)$batch->remaining_quantity, $remainingToDeduct);
            $batch->remaining_quantity -= $take;
            $batch->save();
            $remainingToDeduct -= $take;
        }
    }

    return $sale;
    // If we reach here, ALL steps succeeded → transaction COMMITS
    // If ANY step throws an exception → all changes are automatically ROLLED BACK
});
```

14.3 Exceptions (Throwing and Catching Errors)

```
// THROWING an exception (signaling an error)
if (!$availability['possible']) {
    throw new RuntimeException("Sale rejected: insufficient stock.");
    // This immediately stops execution
}

// CATCHING an exception (handling the error)
try {
    $sale = $this->saleService->recordSale($dishId, $quantity, $userId);
    return redirect()->route('sales.index')->with('success', 'Sale recorded!');
} catch (Exception $e) {
    return back()->withErrors(['stock_error' => $e->getMessage()]);
}
```

Chapter 15: Your Project — Feature-by-Feature Walkthrough

Feature 1: Recipe-Based Inventory Deduction

Step-by-step trace:

```
1. User fills form at /sales/create (selects dish + quantity)
   ↓
2. Browser sends POST /sales with {dish_id: 1, quantity: 2}
   ↓
3. routes/web.php: "POST /sales → SaleController@store"
   ↓
4. SaleController::store()
   - validate: dish_id must exist, quantity must be integer 1-100
   - call: $this->saleService->recordSale(1, 2, $userId)
   ↓
5. SaleService::recordSale()
   - Load dish with recipe: Dish::with('recipeIngredients.ingredient.batches')->findOrFail(1)
   - Check availability: $dish->checkStockAvailability(2)
     → Beef Burger needs: 0.4kg Beef Patty × 2 = 0.8kg, 2 Burger Buns × 2 = 4 Buns
     → Check: Beef Patty usable_stock = 8.0kg ✅
     → Check: Burger Bun usable_stock = 50 ✅
   - DB::transaction { ... }
     → Sale::create({dish_id:1, quantity:2, total_amount:560, ...})
     → SaleDeduction::create({sale_id:1, ingredient_id:2, quantity_deducted:0.8})
     → Ingredient update: current_stock = 8.0 - 0.8 = 7.2
     → Batch FIFO deduction: take 0.8 from oldest batch
   - Return $sale
   ↓
6. SaleController: redirect to sales.index with success message
```

Feature 2: Batch Expiry Monitoring

```
// From Ingredient.php — the key line:
public function getUsableStockAttribute(): float
{
    $expiredStock = $this->batches()
        ->whereNotNull('expiry_date')
        ->where('expiry_date', '<', now()->toDateString())  // ← expired = past today
        ->sum('remaining_quantity');

    return max(0, (float)$this->current_stock - (float)$expiredStock);
}
// EXAMPLE: current_stock = 10kg, expired batch has 3kg → usable = 7kg
```

Feature 3: Automatic Reorder Drafting

```
Logic:
  For each ingredient:
    IF usable_stock ≤ reorder_threshold:
      AND no existing draft PO for this ingredient:
        → Create draft PO with quantity = (target_stock - usable_stock)

Duplicate prevention:
  PurchaseOrder::where('status', 'draft')
    ->whereHas('items', fn($q) => $q->where('ingredient_id', $ingredient->id))
    ->exists()
  → If TRUE: skip (draft already exists)
  → If FALSE: create new draft
```

Feature 4: Purchase-Order Approval

```
Status flow:
  draft → approved (manager clicks Approve)
  draft → rejected (manager clicks Reject with reason)
  approved → visible to suppliers
```

```
public function approve(int $id, Request $request)
{
    $order = PurchaseOrder::findOrFail($id);
    $order->status = 'approved';
    $order->approved_by = Auth::id();
    $order->approved_at = now();
    $order->save();

    return redirect()->route('purchase-orders.index')
        ->with('success', 'Purchase order approved successfully.');
}
```

Feature 5: Physical Stock Reconciliation

```
DATABASE says: 15.0 kg Chicken Breast
STAFF COUNTS:  12.5 kg Chicken Breast (counted physically)
VARIANCE:       -2.5 kg (shortage)

If Manager APPROVES:
  → current_stock updated: 15.0 → 12.5
  → Variance recorded in audit log

If Manager REJECTS:
  → current_stock unchanged
  → Rejection reason saved
```

Feature 6: Waste Reporting & Approval

```
Staff Report:
  Ingredient: Chicken Breast
  Quantity:   2.5 kg (spoiled)
  Reason:     Refrigerator malfunction

System calculates:
  Financial Loss = 2.5 kg × ৳350/kg = ৳875 BDT

If Manager APPROVES:
  → current_stock: 15.0 - 2.5 = 12.5 kg
  → Loss recorded: ৳875 BDT
```

Chapter 16: Viva Q&A Master Sheet

Architecture Questions

Q: Explain your project's MVC architecture.

A: Our project follows strict MVC separation:

- **Models** (`app/Models/`) represent database entities like `Ingredient`, `Sale`, `PurchaseOrder`. They define relationships (a Dish has many RecipeIngredients), accessors (like `usable_stock`), and handle Eloquent ORM operations.
- **Views** (`resources/views/`) are Blade templates that only handle presentation — displaying data, forms, Bootstrap components. They contain zero business logic.
- **Controllers** (`app/Http/Controllers/`) handle HTTP requests, validate input, and coordinate between models and services. They don't contain business logic themselves.

- **Services** (`app/Services/`) — an additional layer we added — contain the domain business logic like stock deduction algorithms and transaction management. This keeps controllers thin and business logic testable in isolation.

Q: What is Eloquent ORM and why did you use it?

A: Eloquent is Laravel's Object-Relational Mapper. Instead of writing raw SQL queries, we work with PHP objects. For example, `Ingredient::find(1)` retrieves an ingredient without writing `SELECT * FROM ingredients WHERE id = 1`. It also handles relationships — `$sale->dish->name` automatically joins the sales and dishes tables. We used it because it prevents SQL injection attacks (all queries are parameterized), reduces boilerplate code, and integrates seamlessly with Laravel's ecosystem.

Q: Why do you have a `Services/` folder? That's not standard Laravel.

A: You're right — it's not enforced by Laravel, but it's a best practice. Without services, our controllers would be hundreds of lines long, mixing HTTP concerns (validation, redirects) with business logic (FIFO batch deduction, stock threshold calculations). The Service pattern keeps controllers thin (20-30 lines) and business logic independently testable. For example, `SaleService::recordSale()` can be unit-tested without simulating an HTTP request.

Database Questions

Q: What is a database transaction and why do you use it?

A: A database transaction is a sequence of operations that either all succeed or all fail together — called **atomicity**. For example, when recording a sale, we must: (1) create the sale record, (2) deduct from ingredient stock, (3) update batch remaining quantities. If any step fails, `DB::transaction` automatically rolls back all previous steps, leaving the database in a consistent state. Without transactions, we could have a sale recorded but stock not deducted.

Q: What does `usable_stock` mean? How is it different from `current_stock`?

A: `current_stock` is the total quantity recorded in the database. However, some of that stock may be in expired batches that can't be used. `usable_stock` is a computed Eloquent accessor in the `Ingredient` model that queries the `ingredient_batches` table, finds all batches where `expiry_date < today()`, sums their `remaining_quantity`, and subtracts from `current_stock`. This implements Feature 2 — expired stock is automatically quarantined from usable inventory without deleting the batch records.

Feature Questions

Q: What is FIFO and how is it implemented?

A: FIFO stands for First In, First Out — the oldest stock is used before newer stock, minimizing spoilage. In our `SaleService`, when deducting ingredients, we query batches ordered by `expiry_date ASC` (soonest expiry first). We loop through batches and take stock from each until the required quantity is met. The SQL ordering is: `orderByRaw('CASE WHEN expiry_date IS NULL THEN 1 ELSE 0 END, expiry_date ASC')` — this puts batches with expiry dates before those without.

Q: Trace the request: "Staff clicks Record Sale for 2 Beef Burgers."

A:

1. Browser sends `POST /sales` with `{dish_id: 1, quantity: 2}`
2. `routes/web.php` matches this to `SaleController@store`
3. `SaleController::store()` validates: dish must exist, quantity 1-100
4. Controller calls `SaleService->recordSale(1, 2, userId)`
5. Service loads Dish with eager-loaded recipe ingredients and their batches
6. Calls `checkStockAvailability(2)` — checks usable stock ≥ required (0.8kg Beef Patty, 4 Burger Buns)
7. If sufficient, opens `DB::transaction`
8. Creates `Sale` record with total ৳560
9. Loops through recipe ingredients; for each: creates `SaleDeduction` audit log, reduces `Ingredient.current_stock`, then deducts from batches in FIFO order
10. Transaction commits; returns Sale object
11. Controller redirects to `sales.index` with success message
12. User sees sales list with "Order recorded" green alert

Q: What happens if stock is insufficient?

A: `SaleService` calls `$dish->checkStockAvailability($quantity)` before opening the transaction. This method checks each recipe ingredient's `usable_stock` against the required quantity. If any ingredient is short, it returns `['possible' => false, 'shortages' => [...]]`. The service then throws a `RuntimeException` with a detailed message. The controller catches this exception and returns to the form with the error displayed. No transaction is opened, so no data is changed.

Q: What is CSRF protection and why does every form have `@csrf`?

A: CSRF stands for Cross-Site Request Forgery. Without protection, a malicious website could trick a logged-in user's browser into submitting a request to our application. Laravel prevents this by generating a unique secret token for each session and embedding it in forms with `@csrf`. When the form is submitted, Laravel verifies the token matches the session. If not, it rejects the request with a 419 error.

Q: Show me the code that prevents expired stock from being sold.

A: Point to `app/Models/Ingredient.php`:

```
public function getUsableStockAttribute(): float
{
    $expiredStock = $this->batches()
        ->whereNotNull('expiry_date')
        ->where('expiry_date', '<', now()->toDateString())
        ->sum('remaining_quantity');

    return max(0, (float)$this->current_stock - (float)$expiredStock);
}
```

"This accessor is called whenever `$ingredient->usable_stock` is accessed. It queries all batches with an expiry date in the past, sums their remaining quantities, and subtracts that from total stock. `max(0, ...)` ensures we never return negative stock. The sale service uses `usable_stock` for availability checking, not `current_stock`."

Q: What does `with(['dish', 'recorder', 'deductions.ingredient'])` do?

A: "This is Eloquent eager loading. When fetching a list of sales, we also want to display the dish name, the recorder's name, and each ingredient that was deducted. Without `with()`, Laravel would run a separate database query for EACH sale to get its dish — if there are 100 sales, that's 101 queries, which is slow. With `with()`, Laravel loads all related data in just 2-3 total queries. The dot notation `'deductions.ingredient'` means: load deductions AND within each deduction, load its ingredient."

💡 PRO TIP: **Final Advice for Your Viva:** 1. Open your project at `http://127.0.0.1:8000` and walk through each feature while explaining it 2. Know your folder structure — be able to say *exactly* which file contains which code 3. When asked "how does X work?" — trace it from the Route → Controller → Service → Model → View 4. It's okay to say "I used AI assistance for scaffolding, but I understand and can explain every line" — your faculty allows this per Section 8 of the guidelines 5. Practice saying each feature in your own words using the restaurant analogy

---

Quick Reference Cheat Sheet

Term	What it means in simple English
Model	PHP class that represents a database table
Controller	Handles incoming web requests
View	HTML template shown to the user
Route	Maps URL to a controller method
Eloquent	ORM — work with database as PHP objects
Migration	PHP code that creates database tables
Seeder	Code that fills database with test data
Artisan	Laravel's command-line tool
Middleware	Code that runs before controller (security gate)
Session	Remembers user between requests
CSRF	Security token required in every form
Transaction	All-or-nothing group of database operations
Eager Loading	Loading related data in one query (`with()`)
Accessor	Computed property on a Model
Service Class	Holds business logic (keeps Controller thin)
Blade	Laravel's HTML template engine (`@foreach` , `@if`)
Namespace	Folder path for PHP classes (`App\Models\...`)
Dependency Injection	Framework automatically creates class instances
Foreign Key	Column linking one table to another table
`$fillable`	Security: list of database fields that can be mass-assigned
`$casts`	Automatically convert database values to PHP types

This guide was written specifically for your CSE470 project: Restaurant Supply Chain and Inventory Hub. Every example comes from your actual project code at `/home/sloth/Projects/cse470_project/` *.*